

VCL2FMXConverter

Delphi Migration Engineering

From VCL structure to FMX execution with disciplined, global conversion rules.

Architecture

Automation

Validation

VCL2FMX CONVERTER V4.1.8

RUNTIME FLOW AND

ARCHITECTURE

Professional runtime flow, architecture, mapping-pack and
verification reference

VCL2FMX Converter is a free, open-source tool designed to assist Delphi developers in migrating VCL projects to FireMonkey. It is intended to reduce migration workload while keeping developers in control of complex platform, database, drawing, and messaging behavior. Always free, never a fee.

Thomas H. Martin

Operational pipeline, mapping-pack flow, conversion stages, reporting, and verification

Version v4.1.8| Updated 2026-06-12

Table of Contents

Foreword	5
v4.1.8 Current Standing.....	7
System at a glance.....	8
1. Startup Sequence.....	9
2. What `FormShow` Does	11
3. Idle Ready State After Startup.....	12
4. Source Folder Selection Flow	13
5. Target Folder Selection Flow.....	14
6. What Happens When the User Clicks Convert.....	15
6.1. It first blocks duplicate runs.....	15
6.2. It reads and trims the source and target edits.....	15
6.3. It validates the source and target paths	15
6.4. It resets the visible run state	16
6.5. It captures the current conversion options.....	16
6.6. It primes the run-summary fields	17
6.7. It disables the interactive controls.....	17
6.8. It creates the worker thread	18
7. What `ConversionThreadProc` Does	20
7.1. It builds the conversion context.....	20
7.2. It logs that the engine is starting	20
7.3. It creates the engine and runs it.....	21
7.4. It captures post-run totals	21
7.5. It decides the final status text	21
8. What the Engine Builds Internally Before the Run.....	23
9. Engine Run Flow: `TConverterEngine.Convert`	24
9.1. It resets internal run state	24
9.2. It initializes live logging	24
9.3. It resets and prepares the file manager	25
10. Mapping-Pack Load Rules	26
Mapping-pack action matrix.....	26
11. Source Scan Rules: What Files Are Collected	27
11.1. The output folder is excluded from scanning	27

11.2. Excluded folders	27
11.3. Recursive vs top-only scan	27
11.4. File-type filtering	28
11.5. File order	28
12. Output Preparation Rules	29
13. File-Processing Loop.....	30
14. Pascal File Flow	31
14.1. The file is read	31
14.2. The orchestrator converts the Pascal code.....	31
14.3. The converted Pascal file is saved	33
15. DFM File Flow	34
DFM pipeline summary	34
15.1. The DFM is loaded	34
15.2. The DFM text is parsed into a component tree	35
15.3. Component analysis happens before FMX output is generated.....	35
15.4. The FMX form text is generated	36
15.5. The output filename is changed from `dfm` to `fmx`	36
16. What Happens After All Source Files Are Converted	38
17. Project Generation Flow	39
17.1. DPR generation	39
17.2. DPROJ generation.....	39
17.3. DEPLOYPROJ generation.....	40
17.4. Companion-file and runtime-support pass	40
18. Output Audit Pass.....	41
19. Report Generation Flow	42
19.1. The report calculates counts and status	42
19.2. The final status is assigned	42
19.3. The text report contents.....	43
19.4. The HTML report contents.....	43
20. Engine Completion Logic	45
21. How the UI Is Restored After the Worker Thread Finishes.....	46
22. What `UpdateArtifactsAfterConversion` Does.....	47
23. What the User Can Do After the Run	48
23.1. Open Output.....	48

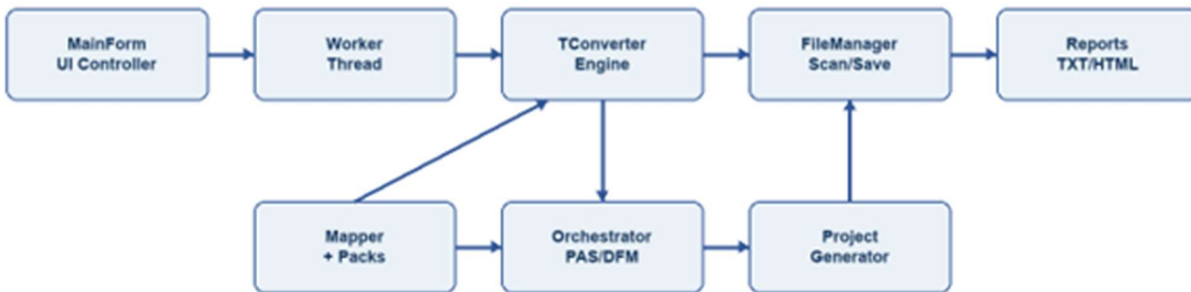
23.2. Open Report	48
23.3. Print Report.....	48
24. Reset Flow	50
25. Close Protection While a Run Is Active	51
26. Thread Boundary Summary	52
26.1. UI thread.....	52
26.2. Worker thread	52
27. Exact Start-to-Finish Narrative in Plain English.....	54

Foreword

The purpose of this document is to explain the complete flow of the VCL2FMX Converter and Migration Assistant. It takes the reader from the moment the application is invoked and the first user activity occurs to the very end when the migration is complete. It will be useful both to the user who converts the applications and also to the software engineers.

This is not a marketing description. It is the true operational flow from program startup through conversion, mapping-pack loading, Pascal and DFM conversion, project generation, report generation, and post-run UI state as of v4.1.8.

VCL2FMX Runtime Architecture



Core idea: the UI starts and finishes the run; the worker thread drives the engine; mapping packs inform DFM generation and reporting.

The current v4.1.8 runtime flow is based on these active source units:

Unit	Runtime responsibility
MainForm.pas	FMX user interface, run setup, thread launch, completion UI, report/output actions.
Converter.Core.Types.pas	Shared context, options, issue records, mapping-pack usage records, constants.
Converter.Core.Engine.pas	Top-level conversion run controller, file loop, report generation, run status.
Converter.Core.FileManager.pas	Source scan, output preparation, relative output paths, encoding-aware saves.
Converter.Core.Integration.pas	Conversion orchestrator coordinating Pascal, DFM, rewrite, and advanced passes.
Converter.Mapper.Component.pas	Built-in mappings, user mappings, JSON mapping-pack loading, property/event inventories.

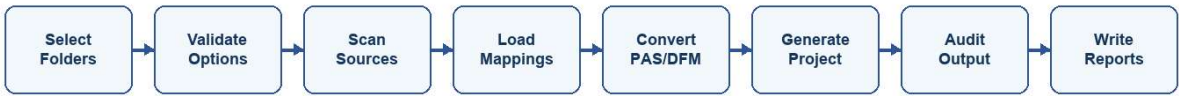
Converter.Parser.Pascal.pas	Pascal parse support and structural analysis.
Converter.Parser.DFM.pas	DFM load/parse/generate pipeline, binary DFM handling, FMX output generation.
Converter.Project.Generator.pas	DPR/DPROJ/deploy/project companion generation and FMX support files.
Advanced and Rewrite units	Specialized conversion passes for WinAPI, data-aware controls, compatibility helpers, LiveBindings, runtime normalization, and uses clauses.

v4.1.8 Current Standing

- The v4.1.8 line is the current v4 development/test line and is designated as a maintenance release.
- The converter is split across 18 active Pascal source units at the project root.
- Mapping packs are enabled by default and are loaded from the mapping_packs folder beside the executable unless the context points elsewhere.
- The current mapping-pack loader validates actions, loads packs deterministically, records loaded packs, and reports pack usage during conversion.
- There are 22 mapping packs included with this software.
- The current acceptance gate passes with 60 checks.

System at a glance

Start-to-Finish Conversion Pipeline



Reports include summary cards, ordinary issues, grouped manual-review items, mapping-pack usage, loaded packs, and next-step guidance.

Layer	Primary unit(s)	Purpose
UI/controller	MainForm.pas	Collect options, start worker thread, show live log, publish completed artifacts.
Engine	Converter.Core.Engine.pas	Run scan/convert/generate/audit/report sequence and final status logic.
Mapping	Converter.Mapper.Component.pas	Resolve classes, properties, events, JSON packs, actions, confidence and source.
DFM conversion	Converter.Parser.DFM.pas	Load binary/text DFM, parse component tree, emit FMX object text.
Pascal conversion	Converter.Core.Integration.pas + rewrite units	Run ordered rewrite pipeline and compatibility/lifecycle helpers.
Reporting	Converter.Core.Engine.pas	Generate text/HTML reports with grouped manual-review and mapping-pack sections.

1. Startup Sequence

When the program launches, the main form is created first. The first major runtime event is ``TfrmMain.FormCreate``.

``FormCreate`` does all of the initial bootstrapping for the user interface and the converter session state.

It explicitly wires runtime event handlers:

1. ``OnCloseQuery := FormCloseQuery``
2. ``OnShow := FormShow``
3. ``OnResize := FormResize``
4. ``TabControlMain.OnChange := TabControlMainChange``

It then initializes the file-type selector:

5. clears the combo box
6. adds ``PAS files only``
7. adds ``DFM files only``
8. adds ``Both PAS and DFM``
9. sets the default selection to ``Both PAS and DFM``

It clears the source and target folder edits:

10. ``edtSource.Text := ```
11. ``edtTarget.Text := ```

It enables the main conversion option checkboxes by default:

12. recursive scanning = on
13. critical areas = on
14. data-aware conversion = on

- 15. third-party handling = on
- 16. WinAPI handling = on

It seeds the live log with the startup instruction text:

`Choose a VCL source folder and an FMX output folder, then start the conversion.`

It makes the built-in reference memos read-only, then generates and loads the reference-page content for:

- 17. component mappings
- 18. property mappings
- 19. event mappings

It then resets all run artifacts and summary fields by calling `ResetArtifacts`, updates the rule summary, marks the UI as not currently converting with

`UpdateUIForConversion(False)`, builds the tab chrome, sets the active tab to `tabProjectScan`, updates the tab chrome again, and applies the responsive layout.

That means that by the time `FormCreate` finishes, the converter is already in a clean "ready for first use" state.

2. What `FormShow` Does

After creation, FMX raises `FormShow`.

`FormShow` does additional startup work that is display-related rather than data-entry-related:

- 20. marks the form as shown
- 21. maximizes the window
- 22. applies responsive layout again
- 23. applies the tab theme
- 24. updates the tab chrome
- 25. ensures the embedded web-browser controls exist for the reference pages
- 26. reloads the component, property, and event reference content into those viewers

One important detail:

`FormCreate` already set the active tab to `tabProjectScan`, so the `FormShow` fallback that would choose `tabDashboard` usually does not change anything. In normal use, the application opens on the Project Scan tab, not the Dashboard tab.

At that point, the form is visible and waits for user activity.

3. Idle Ready State After Startup

Once startup is complete, the program is effectively sitting in a ready state.

The converter is not yet running.

The UI state at this point is:

- 27. source folder is blank
- 28. target folder is blank
- 29. convert options are all on by default
- 30. file-type filter is `Both PAS and DFM`
- 31. status text says the converter is ready
- 32. no report path exists yet
- 33. no output folder path exists yet
- 34. open-report, print-report, and open-output actions are disabled because there is nothing to open yet

4. Source Folder Selection Flow

When the user clicks `Browse Source`, `btnBrowseSourceClick` runs.

This handler:

- 35. copies the current `edtSource.Text` into a local `Dir` variable
- 36. opens `SelectDirectory('Select Source Folder', '', Dir)`
- 37. if the user confirms a folder, it assigns that folder back into `edtSource.Text`
- 38. it then calls `SuggestTargetFromSource(Dir)`

`SuggestTargetFromSource` does not create a folder. It only proposes a default target path in the target edit box.

It works like this:

- 39. if `edtTarget` already contains text, it does nothing
- 40. it strips any trailing delimiter from the source path
- 41. it gets the source folder's parent directory
- 42. it gets the source folder name itself
- 43. it builds:

`<ParentFolder>\<SourceFolderName> - FMX Output`

So if the source is:

`C:\New Delphi Projects\Application7\_2\_4 - Portable`

the suggested target becomes:

`C:\New Delphi Projects\Application7\_2\_4 - Portable - FMX Output`

Important detail:

The target folder is only suggested here. It is not created yet.

5. Target Folder Selection Flow

When the user clicks `Browse Target`, `btnBrowseTargetClick` runs.

This handler:

- 44. copies `edtTarget.Text` into a local `Dir` variable
- 45. opens `SelectDirectory('Select Target Folder', "", Dir)`
- 46. if the user confirms a folder, it writes that folder into `edtTarget.Text`

Again, this does not create the folder yet. It only captures the intended output path.

6. What Happens When the User Clicks Convert

When the user clicks `Convert Project`, `btnConvertClick` runs.

This is the handoff point from the user-facing UI into the real conversion pipeline.

The convert handler performs a strict set of validations before it starts any worker thread.

6.1. It first blocks duplicate runs

If `FConversionThread` is already assigned, it shows:

`A conversion is already running.`

and exits immediately.

6.2. It reads and trims the source and target edits

It stores:

47. `Src := edtSource.Text.Trim`

48. `Dst := edtTarget.Text.Trim`

6.3. It validates the source and target paths

It stops and shows a message if:

49. source folder is blank

50. target folder is blank

51. source folder does not exist

52. source folder and target folder resolve to the same actual path

If source and target are the same, it shows:

`Please choose a different output folder. The source and target folders should not be the same.`

It also switches the tab back to `tabProjectScan` when path validation fails, so the operator returns to the folder-selection screen.

6.4. It resets the visible run state

Once validation passes, it:

53. clears the log

54. adds `Preparing conversion...`

55. switches the active tab to `tabIssues`

That means the user is moved from the selection screen to the live run screen immediately before the worker thread starts.

6.5. It captures the current conversion options

It reads:

56. selected file type

57. recursive on/off

58. critical-areas on/off

59. data-aware on/off

60. third-party on/off

61. WinAPI on/off

62. mapping packs enabled by default, with the folder resolved from the executable-side mapping_packs directory unless overridden in the conversion context

6.6. It primes the run-summary fields

Before the thread starts, it resets all pending counters and display fields:

63. last output path becomes the chosen target path

64. last report paths are cleared

65. pending files processed = 0

66. pending files converted = 0

67. pending files with errors = 0

68. pending files skipped = 0

69. pending manual review count = 0

70. pending blocking count = 0

71. pending duration text = empty

It then updates the dashboard and issues-tab labels so the operator can already see where output and report files are expected to land.

It also calls ``UpdateConversionOutputSummary(True)``, which changes the summary text to:

``Conversion is running. The live log below is updating in real time, and run totals will appear here when the pass finishes.``

6.7. It disables the interactive controls

``UpdateUIForConversion(True)`` disables:

72. Convert

73. Reset

74. Browse Source

- 75. Browse Target
- 76. file-type combo
- 77. all main option checkboxes

It also:

- 78. changes the convert button text to `Converting...`
- 79. sets status text to `Running conversion`
- 80. disables open-report
- 81. disables print-report
- 82. disables open-output

6.8. It creates the worker thread

The actual conversion work does not happen on the UI thread.

`btnConvertClick` creates an anonymous thread and starts it. That thread calls:

`ConversionThreadProc(Src, Dst, FileTypeIndex, Recursive, EnableCritical, EnableDataAware, EnableThirdParty, EnableWinAPI)`

Inside the thread wrapper:

- 83. `StatusText` starts as `Conversion failed`
- 84. the thread runs `ConversionThreadProc`
- 85. regardless of success or failure, it stores:
 - `FPendingCompletionTargetPath := Dst`
 - `FPendingCompletionStatusText := StatusText`
- 86. it then queues `CompleteConversionOnUIThread` back to the main thread using
`TThread.Queue`

So the heavy work is done in the background, but the final UI update is always returned safely to the main thread.

7. What `ConversionThreadProc`` Does

`ConversionThreadProc`` is the thread-side entrypoint for the conversion session.

It creates two major objects:

87. `TConversionContext``

88. `TConverterEngine``

7.1. It builds the conversion context

The context receives the selected run options:

89. `SourcePath``

90. `OutputPath``

91. `ProcessSubdirectories``

92. `CreateReport := True``

93. `EnableCriticalAreas``

94. `EnableDataAware``

95. `EnableThirdParty``

96. `EnableWinAPI``

97. `EnableMappingPacks`` and `MappingPackFolder``

98. file-type filter:

- `ftPas``

- `ftDfm``

- or `ftBoth``

7.2. It logs that the engine is starting

It sends:

`Starting conversion engine...`` to the log.

7.3. It creates the engine and runs it

It instantiates `TConverterEngine``, assigns the screen memo for live log updates, and calls:

```
`Engine.Convert(Context)`
```

7.4. It captures post-run totals

After ``Engine.Convert`` returns, it copies the final counts from the engine and context into the form's pending summary fields:

- 99. files processed
- 100. files converted
- 101. files with errors
- 102. files skipped
- 103. manual review count
- 104. blocking count
- 105. formatted duration text

7.5. It decides the final status text

The final run status shown back in the UI is decided like this:

- 106. if the engine recorded file errors, or the context contains blocking issues:
 - ``Blocking issues present``
- 107. else if the context contains manual-review issues:
 - ``Manual review required``
- 108. else:
 - ``Clean conversion``

If an exception escapes the thread procedure, it shows an error through the synchronized UI path and leaves the final status as:

`Conversion failed`

8. What the Engine Builds Internally Before the Run

`TConverterEngine.Create`` constructs the main subsystems used during a run:

- 109. `TFileManager``
- 110. `TConversionOrchestrator``
- 111. `TProjectGenerator``
- 112. the component mapper loads built-in mappings, user mappings when supplied, and external JSON mapping packs when enabled

Those three objects are the core of the conversion pipeline.

In practical terms:

- 113. the file manager decides what source files are eligible and where output files are saved
- 114. the orchestrator converts Pascal and DFM content
- 115. the project generator creates the output-side project files and copies/stages companion assets

9. Engine Run Flow: `TConverterEngine.Convert`

The engine's `Convert` method is the central pipeline controller.

9.1. It resets internal run state

At the beginning of every run it clears:

- 116. cancellation flag
- 117. processed count
- 118. converted count
- 119. error count
- 120. skipped count
- 121. start time
- 122. context issues
- 123. internal conversion log

It also records the start time into the conversion context.

9.2. It initializes live logging

If no screen memo is attached, it logs that UI updates are disabled.

Normally, because the main form assigned `Engine.ScreenMemo := MemoLog`, the live log is active.

It then writes the conversion header:

- 124. blank line
- 125. `VCL TO FMX CONVERSION STARTED`
- 126. separator line
- 127. source path

128. output path

9.3. It resets and prepares the file manager

The engine calls:

129. `FFileManager.Reset`

130. `FFileManager.PrepareOutput`

These are not the same thing.

`Reset` scans the source side and builds the list of candidate files.

`PrepareOutput` prepares the target side.

10. Mapping-Pack Load Rules

Mapping-pack action matrix

Action	Conversion behavior	Report behavior
convert	Generate the target FMX class when confidence and rule data are sufficient.	Record as third-party/mapping-pack converted usage.
partial	Generate an approximate FMX class and preserve limitations.	Record partial conversion and manual-review note.
manual_review	Do not generate an unsafe replacement.	Group as mapping-pack manual review with reason.
detect_only	Identify vendor/component only.	Group as detection-only; no FMX replacement is emitted.
preserve	Keep original class/reference only where preservation is intentional.	Record preserved component and review guidance.

- 131. Before conversion work relies on mappings, `TComponentMapper.LoadMappings`` loads the built-in mapper database and then calls ``LoadMappingPacks``.
- 132. Mapping packs are optional external JSON files. In v4.1.8 they live in ``mapping_packs`` and are transparent in the text and HTML reports.
- 133. Each pack rule must use a valid action: ``convert``, ``partial``, ``manual_review``, ``detect_only``, or ``preserve``.
- 134. Malformed packs are reported instead of crashing the conversion. Duplicate rules are handled deterministically by sorted file loading and normal mapper precedence.
- 135. During DFM generation, mapping-pack rules can convert, partially convert, preserve, detect only, or force manual review. Each use is recorded in ``FContext.MappingPackUsages`` for report output.
- 136. Low-confidence rules are not silently converted; they are reported for manual review.

11. Source Scan Rules: What Files Are Collected

The file manager starts by clearing its internal file list and resetting its index.

It then verifies that the source directory exists. If it does not, it adds an error issue and stops.

11.1. The output folder is excluded from scanning

If the output folder sits under the source tree, the file manager deliberately skips it so the converter does not re-read its own generated output as input.

11.2. Excluded folders

The file manager excludes any path segment matching:

- 137. `backup`
- 138. `backup copy`
- 139. `win32`
- 140. `win64`
- 141. `deploy`
- 142. `\_\_history`
- 143. `.git`
- 144. `.svn`
- 145. `recovery\_generic`
- 146. `debug`
- 147. `release`

11.3. Recursive vs top-only scan

If recursive mode is on, subdirectories are scanned.

If recursive mode is off, only the top directory is scanned.

11.4. File-type filtering

After a file is found, it must match the currently selected file type:

148. `.pas` only

149. `.dfm` only

150. both `.pas` and `.dfm`

11.5. File order

After collection, the file list is sorted.

That means the converter processes files in sorted order, not raw filesystem discovery order.

12. Output Preparation Rules

`PrepareOutput` is where the converter actually makes the output directory real if it does not exist already.

This is the point where the target folder can be created.

It does the following:

- 151. validates that an output path exists in the options
- 152. if the output folder does not exist, calls `ForceDirectories`
- 153. removes stale build-artifact folders from the output path if they exist:
 - `Win32`
 - `Win64`
 - `Debug`
 - `Release`
 - `\_\_history`

So the output path is not just created. It is also cleaned of the most common stale build subfolders before file conversion begins.

13. File-Processing Loop

After scan and output preparation, the engine asks the file manager for the file count and logs:

` Found X files to process `

It then loops while:

154. more files exist

155. the run has not been cancelled

For each file it:

156. gets the next full file path

157. increments the processed counter

158. logs `[n/total] Processing: filename`

159. routes the file by extension:

- `.pas` -> `ProcessPasFile`

- `.dfm` -> `ProcessDfmFile`

If an exception occurs around a file-level pass, the engine:

160. increments file-error count

161. logs a failed message

162. adds a blocking issue to the context

14. Pascal File Flow

When a Pascal file is processed, this is the actual sequence.

14.1. The file is read

The engine:

- 163. logs that it is processing the Pascal file
- 164. sets ``FContext.CurrentFile``
- 165. tries to read the file using encoding detection and fallback logic

If the read fails:

- 166. the file is counted as skipped
- 167. a failure is logged
- 168. no conversion is attempted

14.2. The orchestrator converts the Pascal code

If reading succeeds, the engine calls:

``FOrchestrator.ConvertPascal(AFileName, Code)``

Inside the orchestrator, the Pascal pass runs in this order:

Step	Stage	Purpose
1	Parse Pascal	Deep-parse the unit and extract method structure.
2	Process method bodies	Handle message, GDI, synchronize, and similar patterns.
3	Automatic fixes	Apply safe early rewrites before advanced converters.
4	Structural wrapper sync	Keep wrapper panels aligned with DFM-side TLayout mapping.
5	Advanced converters	Run critical-area, data-aware, third-party, WinAPI, and graphics passes.
6	Second automatic fixes	Sanitize generated compatibility code; pass is intentional and idempotent.
7-9	Runtime graphics/text/data prep	Wrap canvas scenes, normalize colors, promote memo fields.
10	LiveBindings preflight	Warn about uncommon layouts, then inject generated bindings.
11-13	Compatibility/lifecycle	Reduce cleanup noise and inject compatibility/lifecycle helpers.

14-16	Structure and uses cleanup	Normalize resource directive, check implementation, remove duplicate implementation uses.
17-20	Late runtime/final cleanup	Root double-click adapters, text-layout math, helper reinjection, guarded final uses cleanup.

That is the real Pascal conversion chain in the live v4.1.8 code.

14.3. The converted Pascal file is saved

If conversion succeeds:

- 169. the engine increments `FilesConverted`
- 170. the file manager computes the output file name using the path relative to the source root
- 171. it creates any needed subdirectories in the output tree
- 172. it sanitizes invalid control characters from the output text
- 173. it writes the file while preserving the source encoding style as closely as possible
- 174. it adds an informational issue describing what was saved

If conversion fails:

- 175. the engine increments file errors
- 176. logs the failure
- 177. adds a warning or error issue to the context
- 178. does not save the converted file

15. DFM File Flow

DFM pipeline summary

Stage	Behavior
Load	Use parser-side `LoadDFM`, not generic text reading.
Binary detection	Detect modern resource-header binary DFM and raw `TPF0` streams.
Text conversion	Try binary/resource conversion paths and fail explicitly instead of saving empty FMX.
Parse	Build a component tree with properties, events, collections, subcomponents, and line numbers.
Analyze	Run third-party and data-aware analysis according to selected options.
Generate	Apply built-in and mapping-pack rules, validate actions/confidence, and emit FMX text.
Save	Write `.fmx` output in relative position under the target folder.

When a DFM file is processed, this is the actual sequence.

15.1. The DFM is loaded

The engine logs that it is processing the DFM and sets `FContext.CurrentFile`.

Inside the orchestrator, the DFM parser loads the file through `LoadDFM`.

If the DFM is binary:

- 179. it detects binary DFM streams using Delphi stream-format detection, including modern resource-header forms and raw `TPF0` streams
- 180. converts binary DFM to text with the parser-side binary conversion path, trying the resource and binary stream conversion routes before failing explicitly

If the DFM is already text:

- 181. it uses the shared safe text-read path and tries UTF-8
- 182. falls back to ANSI when needed

15.2. The DFM text is parsed into a component tree

The parser:

- 183. clears the old component list
- 184. resets the current component and collection state
- 185. walks the DFM line by line
- 186. parses `object` / `inherited` declarations
- 187. parses collections and subcomponents
- 188. parses properties and events
- 189. records original line numbers for properties and events

The result is an internal tree of `TDFMComponent` objects describing the source form.

15.3. Component analysis happens before FMX output is generated

The orchestrator then:

- 190. recursively analyzes components for third-party detection if third-party handling is enabled

- 191. runs data-aware analysis if data-aware handling is enabled
- 192. clears old data-binding metadata if data-aware handling is disabled

15.4. The FMX form text is generated

The parser then converts the parsed DFM tree into FMX text by calling:

```
`FDfmParser.Convert(FComponentMapper)`
```

The FMX generator:

- 193. takes the parsed root component
- 194. generates the FMX object tree
- 195. uses the component mapper to translate VCL classes, properties, and events
- 196. records third-party and mapping-pack usage for the final report
- 197. honors mapping-pack actions such as convert, partial, manual_review, detect_only, and preserve
- 198. normalizes generated FMX values such as align constants
- 199. emits FMX text for the output form

After that, the orchestrator applies the full-text memo promotion pass to the generated FMX text and replaces the original DFM text with the generated FMX version.

It then logs:

```
`DFM conversion successful: <file>`
```

15.5. The output filename is changed from `.dfm` to `.fmx`

When the file manager saves a converted DFM, it automatically changes the output extension from `.dfm` to `.fmx`.

That means:

`SomeForm.dfm`

becomes:

`SomeForm.fmx`

in the FMX output tree.

16. What Happens After All Source Files Are Converted

After the file loop finishes, the engine logs:

```
`Generating project files...`
```

It then runs:

```
200.  `FProjectGenerator.GenerateProject`
```

```
201.  `AuditGeneratedOutput`
```

```
202.  `GenerateReport`
```

in that order.

17. Project Generation Flow

``TProjectGenerator.GenerateProject`` performs four major actions:

- 203. ``CopyDPR``
- 204. ``CopyDPROJ``
- 205. ``CopyDeployProj``
- 206. ``CopyProjectCompanionFiles``

It then adds an informational issue:

``Delphi project files generated for FMX output``

17.1. DPR generation

The generator:

- 207. finds the original `` .dpr``
- 208. reads it
- 209. transforms the startup content
- 210. writes the transformed `` .dpr`` into the FMX output folder as UTF-8

17.2. DPROJ generation

The generator:

- 211. finds the original `` .dproj``
- 212. reads it
- 213. transforms project metadata and namespaces
- 214. writes the transformed `` .dproj`` into the FMX output folder as UTF-8

17.3. DEPLOYPROJ generation

If a `.deployproj`` exists, it is likewise found, transformed, and written into the output folder.

17.4. Companion-file and runtime-support pass

The project generator then:

- 215. copies root-level project companion files such as `.res``, `.ico``, `.manifest``, `.png``, and `.bmp``
- 216. inspects project metadata for additional referenced assets
- 217. stages referenced assets into the output tree when possible
- 218. records warnings when referenced assets cannot be copied
- 219. scans the source tree for runtime-support files such as `.exe``, `.dll``, `.lib``, and `.sf2``
- 220. identifies stageable runtime companions such as `.exe``, `.dll``, and `.sf2``
- 221. stages selected runtime companions into the FMX output directory
- 222. records informational notes in the report about what was staged and what still requires deployment review

18. Output Audit Pass

After project generation, the engine audits the generated output.

This pass walks the output tree looking at:

223. `.pas`

224. `.dpr`

225. `.fmx`

Important detail:

It only audits files touched during the current run. That is done by comparing file write times against the run start time. This is specifically meant to avoid stale leftovers in reused output folders from polluting the report for the current conversion.

19. Report Generation Flow

If report creation is enabled, the engine generates both:

- 226. `VCL\_to\_FMX\_Conversion\_Report.txt`
- 227. `VCL\_to\_FMX\_Conversion\_Report.html`

Both are written directly into the output folder root.

19.1. The report calculates counts and status

The report pass calculates:

- 228. informational issue count
- 229. warning count
- 230. manual-review group count
- 231. error count
- 232. blocking-item count
- 233. total visible issue count
- 234. distinct files needing attention

19.2. The final status is assigned

The final status text inside the report is:

- 235. `Blocking issues present` if there were file errors or blocking issues
- 236. `Manual review required` if there were manual-review items but no blocking outcome
- 237. `Clean conversion` otherwise

19.3. The text report contents

The text report includes:

- 238. conversion summary
- 239. total time
- 240. files processed
- 241. files converted
- 242. files with errors
- 243. files skipped
- 244. total issues
- 245. distinct files needing attention
- 246. final status
- 247. manual fixes required
- 248. blocking items
- 249. files with conversion errors
- 250. recommended next step
- 251. mapping packs loaded and mapping-pack usage summaries
- 252. detailed issues
- 253. grouped manual-review section
- 254. issue summary table

19.4. The HTML report contents

The HTML report contains the same logical data in a styled HTML dashboard format, including:

- 255. hero summary/status area
- 256. metric cards
- 257. detailed issue cards
- 258. grouped manual-review cards
- 259. issue summary table
- 260. recommended next-step guidance
- 261. mapping-pack summary sections and grouped mapping-pack manual-review/detect-only cards

20. Engine Completion Logic

After the report is written, the engine decides how to log completion.

If blocking issues exist:

262. it logs a message saying blocking issues are present

263. it returns ``False``

If there are no blocking issues but manual-review issues exist:

264. it logs that manual review is required

265. it returns ``True``

If neither blocking nor manual-review issues exist:

266. it logs that conversion completed cleanly

267. it returns ``True``

Important detail:

The worker-thread status string shown back in the main form is decided separately in ``ConversionThreadProc``, but it uses the same basic logic.

21. How the UI Is Restored After the Worker Thread Finishes

When the background thread ends, it does not directly update the UI.

Instead it queues `CompleteConversionOnUIThread``.

That method runs back on the main thread and does exactly three things:

- 268. frees the conversion thread object
- 269. calls `UpdateUIForConversion(False)`` to re-enable controls
- 270. calls `UpdateArtifactsAfterConversion(TargetPath, StatusText)``

22. What `UpdateArtifactsAfterConversion` Does

This is the method that makes the finished run visible to the operator.

It:

- 271. marks that a run has completed
- 272. copies all pending counters into the last-run fields
- 273. stores the last output path
- 274. computes the expected text and HTML report paths
- 275. picks the preferred report path, preferring HTML over text
- 276. updates the status labels
- 277. updates dashboard output and report labels
- 278. updates issues-tab output and report labels
- 279. rebuilds the summary sentence
- 280. re-enables open-report, print-report, and open-output actions as appropriate

So this is the moment when the UI leaves the "running" state and becomes a post-run review screen.

23. What the User Can Do After the Run

Once conversion is complete and the UI is unlocked, the operator can:

- 281. open the output folder
- 282. open the preferred report
- 283. print the preferred report
- 284. reset the converter back to its startup state

23.1. Open Output

``btnOpenOutputClick``:

- 285. checks whether ``FLastOutputPath`` exists
- 286. if not, shows ``There is no converted output folder to open yet.``
- 287. otherwise uses Windows ``ShellExecute('open', path)`` to open the folder

23.2. Open Report

``btnOpenReportClick``:

- 288. asks for the preferred report path
- 289. prefers HTML if it exists
- 290. otherwise uses the text report if it exists
- 291. if neither exists, shows ``There is no generated report to open yet.``
- 292. otherwise opens the report with ``ShellExecute('open', path)``

23.3. Print Report

``btnPrintReportClick``:

- 293. resolves the preferred report path
- 294. if none exists, shows `There is no generated report to print yet.`
- 295. attempts `ShellExecute('print', reportPath, hideWindow=True)`
- 296. if direct print is not available, it opens the report instead and tells the user that
direct printing was not available

24. Reset Flow

If the user clicks `Reset`, `btnResetClick` runs.

If a conversion thread is still active, reset is blocked with:

`Please wait for the current conversion to finish before resetting.`

If no conversion is running, `ResetToStartupState` is called.

That method:

- 297. clears source and target edits
- 298. restores file type to `Both PAS and DFM`
- 299. re-enables all main option checkboxes to their default checked state
- 300. clears pending completion and message fields
- 301. restores the startup log text
- 302. rebuilds and reloads the reference-page content
- 303. resets artifacts and run summary
- 304. updates the rule summary
- 305. marks the UI as not converting
- 306. scrolls the tab scroll boxes back to the top
- 307. returns the active tab to `tabProjectScan`
- 308. applies responsive layout
- 309. moves focus back to the source edit if possible

So reset is a full UI-state reset, not just a log clear.

25. Close Protection While a Run Is Active

If the user tries to close the program while a conversion is still running, ``FormCloseQuery`` blocks the close.

It only allows the program to close when ``FConversionThread`` is not assigned.

If a run is active, it shows:

``A conversion is still running. Please wait for it to finish before closing the program.``

That means the converter deliberately prevents shutdown in the middle of an active conversion pass.

26. Thread Boundary Summary

There are two major runtime domains in the program:

26.1. UI thread

The UI thread handles:

- 310. form startup
- 311. folder selection
- 312. validation
- 313. starting the worker thread
- 314. log display updates
- 315. synchronized error dialogs
- 316. final completion UI updates
- 317. post-run actions like open-report, print-report, open-output, and reset

26.2. Worker thread

The worker thread handles:

- 318. conversion context creation
- 319. engine construction
- 320. source scan
- 321. output preparation
- 322. per-file Pascal conversion
- 323. per-file DFM conversion
- 324. project generation
- 325. generated-output audit
- 326. report generation
- 327. final run-status determination

The worker thread never directly finalizes the UI. It hands completion back through `TThread.Queue``.

27. Exact Start-to-Finish Narrative in Plain English

If you want the whole thing in one straight-through human narrative, this is it:

328. The program starts.
329. ``FormCreate`` runs.
330. The form wires its runtime events, loads the file-type options, defaults all major conversion modules to on, loads the mapping-reference material, resets all run artifacts, and sets the active tab to Project Scan.
331. ``FormShow`` runs.
332. The window maximizes, the theme/layout pass runs, and the embedded reference browsers are prepared.
333. The form is displayed and waits for user activity.
334. The user chooses a source folder.
335. The converter writes that folder into ``edtSource``.
336. If the target folder is still blank, the converter suggests a sibling folder named ``<SourceName> - FMX Output``.
337. The user can accept that target or browse for another target folder.
338. Nothing is created yet just from selecting folders.
339. The user clicks ``Convert Project``.
340. The converter validates source and target.
341. It rejects blank paths, missing source folders, or a source path equal to the target path.
342. If validation succeeds, it clears the log, switches to the Issues tab, resets pending counters, updates output/report labels, disables interactive controls, changes the button text to ``Converting...``, and starts a worker thread.
343. The worker thread creates a conversion context and copies in all selected options, including mapping-pack settings.
344. The worker thread creates the engine.
345. The engine resets its counters and issues, loads built-in/user/pack mappings through the mapper path, and logs the start banner.
346. The file manager scans the source tree, skipping excluded directories and skipping the output tree if it sits under the source tree.
347. The output directory is created if needed, and stale build subfolders under the output folder are deleted.
348. The engine loops through the sorted list of eligible `` .pas`` and `` .dfm`` files.

349. Each Pascal file is read, parsed, transformed through the orchestrator's multi-step pipeline, and saved into the output tree in relative position.
350. Each DFM file is loaded through the parser, converted from binary to text if needed, parsed into components, translated into FMX text with built-in and mapping-pack rules, renamed to `.fmx``, and saved into the output tree in relative position.
351. After source files finish, the project generator creates transformed `.dpr``, `.dproj``, and `.deployproj`` files and copies or stages project assets and runtime companion files.
352. The engine audits the generated output that was touched during the current run.
353. The engine writes both the text and HTML conversion reports into the output folder, including loaded mapping packs, mapping-pack usage, grouped manual-review items, and ordinary issues.
354. The engine decides whether the run was clean, manual-review, or blocking.
355. The worker thread stores the final status and queues the completion handler back to the main thread.
356. The UI thread receives that completion callback.
357. The UI re-enables all controls, stores the final output and report paths, updates the dashboard and issues panels, and enables the report/output actions.
358. The user can now open the output folder, open the HTML report, print the report, inspect the generated FMX project, or reset the converter for another run.